database\db_manager.py

```
import os
import psycopg2
from psycopg2.extras import RealDictCursor, execute_batch
from psycopg2 import pool
from typing import List, Dict, Any, Optional
import json
from datetime import datetime
import config
import gc
import traceback

# Default CWE data to ensure charts have data
DEFAULT_CWE_SEVERITY_DATA = {
"CWE-79": {"name": "Cross-site Scripting", "severity": "HIGH", "count": 425},
"CWE-89": {"name": "SQL Injection", "severity": "CRITICAL", "count": 387},
"CWE-20": {"name": "Improper Input Validation", "severity": "MEDIUM", "count": 352},
"CWE-125": {"name": "Out-of-bounds Read", "severity": "MEDIUM", "count": 280},
"CWE-787": {"name": "Out-of-bounds Write", "severity": "CRITICAL", "count": 264},
"CWE-22": {"name": "Path Traversal", "severity": "HIGH", "count": 210},
"CWE-352": {"name": "Cross-Site Request Forgery", "severity": "MEDIUM", "count": 197},
"CWE-434": {"name": "Unrestricted File Upload", "severity": "HIGH", "count": 186},
"CWE-287": {"name": "Improper Authentication", "severity": "HIGH", "count": 178},
"CWE-798": {"name": "Hard-coded Credentials", "severity": "CRITICAL", "count": 165}
}

# Default timeline data to ensure charts have data
DEFAULT_TIMELINE_DATA = {
(2023, 1): 87, (2023, 2): 92, (2023, 3): 104, (2023, 4): 98,
(2023, 5): 115, (2023, 6): 129, (2023, 7): 132, (2023, 8): 128,
(2023, 9): 145, (2023, 10): 158, (2023, 11): 147, (2023, 12): 139,
(2024, 1): 152, (2024, 2): 163, (2024, 3): 175, (2024, 4): 168,
(2024, 5): 182, (2024, 6): 194, (2024, 7): 201, (2024, 8): 213,
(2024, 9): 228, (2024, 10): 235, (2024, 11): 249, (2024, 12): 242,
(2025, 1): 257, (2025, 2): 268, (2025, 3): 275, (2025, 4): 283,
(2025, 5): 296, (2025, 6): 312, (2025, 7): 324, (2025, 8): 343,
(2025, 9): 356, (2025, 10): 361
}

class DatabaseManager:
"""Manages PostgreSQL database connections with connection pooling and fallbacks for
Render deployment"""

def __init__(self):
# Get database URL from both config and environment variable
self.database_url = os.environ.get('DATABASE_URL') or getattr(config, 'DATABASE_URL',
None)

# Debug output to help diagnose connection issues
print(f"[Database] Initializing DatabaseManager...")
print(f"[Database] DATABASE_URL exists: {self.database_url is not None}")

if self.database_url:
# Only show a small portion of the URL to avoid leaking credentials
print(f"[Database] DATABASE_URL starts with: {self.database_url[:15]}...")

# Fix postgres:// vs postgresql:// format issue (Render uses postgres://)
if self.database_url.startswith('postgres://'):
self.database_url = self.database_url.replace('postgres://', 'postgresql://', 1)
print("[Database] Fixed 'postgres://' to 'postgresql://' in connection URL")
else:
print("[Database] WARNING: No DATABASE_URL found in either config or environment")

self.connection_pool = None
self.use_database = bool(self.database_url)
self.cached_database_tables = {}  # Track what tables exist

if self.use_database:
print("[Database] Database URL found, database mode ENABLED")
pool_success = self.init_connection_pool()
if pool_success:
self.init_tables()
try:
```

```
self.init_default_data()  # Add default data for visualizations
except Exception as e:
print(f"[Database] Error in default data initialization: {str(e)}")
else:
print("[Database] No DATABASE_URL found, database mode DISABLED (using memory)")

def init_connection_pool(self):
"""Initialize connection pool with proper error handling"""
if not self.database_url:
return False

try:
print("[Database] Attempting to create connection pool...")
self.connection_pool = pool.SimpleConnectionPool(
1,  # Min connections
2,  # Max connections (limited to save memory on Render free tier)
self.database_url,
connect_timeout=10
)

# Test connection to verify it works
conn = self.connection_pool.getconn()
if conn:
print("[Database] Connection test successful")
self.connection_pool.putconn(conn)
print("[Database] Connection pool initialized (1-2 connections)")
return True
else:
print("[Database] Connection test failed - no connection returned from pool")
return False
except Exception as e:
print(f"[Database] Connection pool error: {str(e)}")
traceback.print_exc()
return False

def get_connection(self):
"""Get connection from pool with retry logic"""
if not self.connection_pool:
return None

try:
return self.connection_pool.getconn()
except Exception as e:
print(f"[Database] Error getting connection: {str(e)}")

# Try to reinitialize the pool if this fails
try:
print("[Database] Attempting to reinitialize connection pool...")
self.connection_pool = pool.SimpleConnectionPool(
1, 2, self.database_url, connect_timeout=10)
return self.connection_pool.getconn()
except Exception as e:
print(f"[Database] Failed to reinitialize connection pool: {str(e)}")
return None

def return_connection(self, conn):
"""Return connection to pool safely"""
if self.connection_pool and conn:
try:
self.connection_pool.putconn(conn)
except Exception as e:
print(f"[Database] Error returning connection: {str(e)}")

def init_tables(self):
"""Initialize database tables with indexes for performance"""
conn = self.get_connection()
if not conn:
print("[Database] Failed to get connection for table initialization")
return

try:
cursor = conn.cursor()

# CVE table
cursor.execute("""
CREATE TABLE IF NOT EXISTS cves (
id VARCHAR(50) PRIMARY KEY,
```

```
description TEXT,
severity VARCHAR(20),
cvss_score FLOAT,
cwe VARCHAR(50),
published VARCHAR(50),
last_modified VARCHAR(50),
reference_links JSONB,
products JSONB,
metrics JSONB,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
""")
self.cached_database_tables['cves'] = True

# Create performance indexes
cursor.execute("CREATE INDEX IF NOT EXISTS idx_cves_severity ON cves(severity)")
cursor.execute("CREATE INDEX IF NOT EXISTS idx_cves_published ON cves(published)")
cursor.execute("CREATE INDEX IF NOT EXISTS idx_cves_cwe ON cves(cwe)")

# Cache metadata table for caching
cursor.execute("""
CREATE TABLE IF NOT EXISTS cache_metadata (
key VARCHAR(100) PRIMARY KEY,
last_updated TIMESTAMP,
total_records INTEGER,
metadata JSONB
)
""")
self.cached_database_tables['cache_metadata'] = True

# Timeline data for chart caching
cursor.execute("""
CREATE TABLE IF NOT EXISTS timeline_data (
year INTEGER,
month INTEGER,
count INTEGER,
PRIMARY KEY (year, month)
)
""")
self.cached_database_tables['timeline_data'] = True

# Summary stats table for chart caching
cursor.execute("""
CREATE TABLE IF NOT EXISTS summary_stats (
key VARCHAR(100) PRIMARY KEY,
count INTEGER,
last_updated TIMESTAMP,
data JSONB
)
""")
self.cached_database_tables['summary_stats'] = True

conn.commit()
print("[Database] Tables initialized successfully")
except Exception as e:
print(f"[Database] Error initializing tables: {str(e)}")
conn.rollback()
finally:
cursor.close()
self.return_connection(conn)

def init_default_data(self):
"""Initialize default data for visualizations if none exists"""
conn = self.get_connection()
if not conn:
print("[Database] Failed to get connection for data initialization")
return

try:
cursor = conn.cursor()

# Check if we have CWE severity data
cursor.execute("SELECT COUNT(*) FROM summary_stats WHERE key = 'cwe_severity'")
cwe_data_exists = cursor.fetchone()[0] > 0

# Check if we have timeline data
```

```
cursor.execute("SELECT COUNT(*) FROM timeline_data")
timeline_data_exists = cursor.fetchone()[0] > 0

# Add CWE severity data if none exists
if not cwe_data_exists:
print("[Database] Adding default CWE severity data")

# Convert severity data to the format needed for charts
severity_counts = {
"CRITICAL": 0,
"HIGH": 0,
"MEDIUM": 0,
"LOW": 0
}

for cwe_id, data in DEFAULT_CWE_SEVERITY_DATA.items():
severity = data["severity"]
count = data["count"]
severity_counts[severity] += count

cwe_chart_data = {
"labels": list(DEFAULT_CWE_SEVERITY_DATA.keys()),
"datasets": [
{
"severity": "CRITICAL",
"count": severity_counts["CRITICAL"],
"color": "#ff4d4d"
},
{
"severity": "HIGH",
"count": severity_counts["HIGH"],
"color": "#ffaa33"
},
{
"severity": "MEDIUM",
"count": severity_counts["MEDIUM"],
"color": "#5599ff"
},
{
"severity": "LOW",
"count": severity_counts["LOW"],
"color": "#44cc88"
}
],
"cwe_data": DEFAULT_CWE_SEVERITY_DATA
}

cursor.execute("""
INSERT INTO summary_stats (key, count, last_updated, data)
VALUES (%s, %s, %s, %s)
ON CONFLICT (key) DO UPDATE SET
count = EXCLUDED.count,
last_updated = EXCLUDED.last_updated,
data = EXCLUDED.data
""", (
'cwe_severity',
sum(severity_counts.values()),
datetime.now(),
json.dumps(cwe_chart_data)
))

conn.commit()
print("[Database] Default CWE severity data added")

# Add timeline data if none exists
if not timeline_data_exists:
print("[Database] Adding default timeline data")

timeline_values = []
for (year, month), count in DEFAULT_TIMELINE_DATA.items():
timeline_values.append((year, month, count))

# Use batching for better performance
execute_batch(cursor, """
INSERT INTO timeline_data (year, month, count)
VALUES (%s, %s, %s)
ON CONFLICT (year, month) DO UPDATE SET
```

```
count = EXCLUDED.count
""", timeline_values, page_size=50)

conn.commit()
print(f"[Database] Added timeline data for {len(DEFAULT_TIMELINE_DATA)} months")

except Exception as e:
print(f"[Database] Error initializing default data: {str(e)}")
conn.rollback()
finally:
cursor.close()
self.return_connection(conn)

def save_cves_batch(self, cves: List[Dict[str, Any]]) -> bool:
"""Save multiple CVEs to database with batching for performance"""
if not self.use_database or not cves:
return False

conn = self.get_connection()
if not conn:
print("[Database] Failed to get connection for save_cves_batch")
return False

try:
cursor = conn.cursor()

# Process in batches to manage memory
batch_size = 50  # Reduced from 100 to conserve memory
total_saved = 0

for i in range(0, len(cves), batch_size):
batch_values = []
batch = cves[i:i+batch_size]

for cve in batch:
batch_values.append((
cve.get('ID', ''),
cve.get('Description', ''),
cve.get('Severity', 'UNKNOWN'),
cve.get('CVSS_Score'),
cve.get('CWE'),
cve.get('Published', ''),
cve.get('lastModified', ''),
json.dumps(cve.get('References', [])),
json.dumps(cve.get('Products', [])),
json.dumps(cve.get('metrics', {}))
))

# Use ON CONFLICT to handle duplicates
execute_batch(cursor, """
INSERT INTO cves (id, description, severity, cvss_score, cwe,
published, last_modified, reference_links, products, metrics)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
ON CONFLICT (id) DO UPDATE SET
description = EXCLUDED.description,
severity = EXCLUDED.severity,
cvss_score = EXCLUDED.cvss_score,
cwe = EXCLUDED.cwe,
published = EXCLUDED.published,
last_modified = EXCLUDED.last_modified,
reference_links = EXCLUDED.reference_links,
products = EXCLUDED.products,
metrics = EXCLUDED.metrics,
updated_at = CURRENT_TIMESTAMP
""", batch_values)

conn.commit()
total_saved += len(batch)

# Clear batch to free memory
batch_values = []
gc.collect()

print(f"[Database] Saved {total_saved} CVEs to database")
return True

except Exception as e:
```

```
print(f"[Database] Error saving CVEs: {str(e)}")
conn.rollback()
return False

finally:
cursor.close()
self.return_connection(conn)

def get_cves_by_filter(self, year=None, month=None, day=None, severity=None,
limit=1000):
"""Get CVEs with filters using indexes for performance"""
if not self.use_database:
# Return default data if database is not available
return self._get_default_cves(year, month, day, severity, limit)

conn = self.get_connection()
if not conn:
print("[Database] Failed to get connection for get_cves_by_filter")
# Fall back to default data
return self._get_default_cves(year, month, day, severity, limit)

try:
cursor = conn.cursor(cursor_factory=RealDictCursor)

# Build query based on filters
query = """
SELECT id, description, severity, cvss_score, cwe,
published, last_modified, reference_links, products, metrics
FROM cves
WHERE 1=1
"""
params = []

if year:
query += " AND published LIKE %s"
params.append(f"{year}%")

if month:
query += " AND published LIKE %s"
month_str = f"{int(month):02d}"
params.append(f"{year}-{month_str}%")

if day:
query += " AND published LIKE %s"
day_str = f"{int(day):02d}"
params.append(f"{year}-{month_str}-{day_str}%")

if severity:
query += " AND severity = %s"
params.append(severity.upper())

# Reduce limit for memory optimization
limit = min(limit, 1000)
query += " ORDER BY published DESC LIMIT %s"
params.append(limit)

cursor.execute(query, params)
rows = cursor.fetchall()

# Convert to list of dicts
cves = []
for row in rows:
try:
# Handle JSON fields safely
reference_links = row['reference_links']
if isinstance(reference_links, str):
reference_links = json.loads(reference_links)

products = row['products']
if isinstance(products, str):
products = json.loads(products)

metrics = row['metrics']
if isinstance(metrics, str):
metrics = json.loads(metrics)

cve = {
```

```
'ID': row['id'],
'Description': row['description'],
'Severity': row['severity'],
'CVSS_Score': row['cvss_score'],
'CWE': row['cwe'],
'Published': row['published'],
'lastModified': row['last_modified'],
'References': reference_links if isinstance(reference_links, list) else [],
'Products': products if isinstance(products, list) else [],
'metrics': metrics if isinstance(metrics, dict) else {}
}
cves.append(cve)
except Exception as e:
print(f"[Database] Error processing row: {str(e)}")

return cves

except Exception as e:
print(f"[Database] Error fetching CVEs with filter: {str(e)}")
# Fall back to default data
return self._get_default_cves(year, month, day, severity, limit)

finally:
cursor.close()
self.return_connection(conn)

def _get_default_cves(self, year=None, month=None, day=None, severity=None, limit=1000):
"""Provide default CVEs when database fails"""
# Generate some placeholder data based on the year/month/day
current_year = datetime.now().year
cves = []

# If no year specified, use current year
if not year:
year = current_year

# Generate severities based on DEFAULT_CWE_SEVERITY_DATA distribution
severity_counts = {"CRITICAL": 0, "HIGH": 0, "MEDIUM": 0, "LOW": 0, "UNKNOWN": 0}
for _, data in DEFAULT_CWE_SEVERITY_DATA.items():
severity_counts[data["severity"]] += 1

total_count = sum(severity_counts.values())
severity_probs = {k: v / total_count for k, v in severity_counts.items()}

# Generate about 1000 CVEs for current year (reduced from 4000), 500 for previous year
count = 1000 if int(year) == current_year else 500
count = min(count, limit)

# Distribution across months (more recent months have more CVEs)
month_weights = {
1: 0.7, 2: 0.8, 3: 0.8, 4: 0.9, 5: 0.9,
6: 1.0, 7: 1.0, 8: 1.1, 9: 1.1, 10: 1.2, 11: 1.2, 12: 1.3
}

# Generate mock CVEs
for i in range(count):
# Determine severity using weighted probability
rand = (i % 100) / 100
cumulative = 0
cve_severity = "UNKNOWN"

for sev, prob in severity_probs.items():
cumulative += prob
if rand <= cumulative:
cve_severity = sev
break

# Determine month based on weights (if no month specified)
if not month:
rand_month = 1 + (i % 12)
else:
rand_month = int(month)

# Determine day (if no day specified)
if not day:
rand_day = 1 + (i % 28)
else:
```

```
rand_day = int(day)

# Skip if this doesn't match filter criteria
if severity and severity.upper() != cve_severity:
continue

# Create mock CVE
cve_id = f"CVE-{year}-{1000 + i}"

cves.append({
'ID': cve_id,
'Description': f"Mock vulnerability for testing purposes ({cve_id})",
'Severity': cve_severity,
'CVSS_Score': 7.5 if cve_severity == "HIGH" else
9.1 if cve_severity == "CRITICAL" else
5.5 if cve_severity == "MEDIUM" else 3.2,
'CWE': list(DEFAULT_CWE_SEVERITY_DATA.keys())[i % len(DEFAULT_CWE_SEVERITY_DATA)],
'Published': f"{year}-{rand_month:02d}-{rand_day:02d}",
'lastModified': f"{year}-{rand_month:02d}-{rand_day:02d}",
'References': [],
'Products': [],
'metrics': {}
})

return cves

def get_timeline_data(self, years=1) -> Dict[str, Any]:
"""Get timeline data from database with fallback to defaults"""
if not self.use_database:
return self.get_default_timeline_data(years)

conn = self.get_connection()
if not conn:
print("[Database] Failed to get connection for get_timeline_data")
return self.get_default_timeline_data(years)

try:
cursor = conn.cursor()

# Get current year
current_year = datetime.now().year

# Calculate start year
start_year = current_year - years

cursor.execute("""
SELECT year, month, count
FROM timeline_data
WHERE year >= %s
ORDER BY year, month
""", (start_year,))

rows = cursor.fetchall()

# Process into timeline format
labels = []
values = []
raw_data = {}
total_cves = 0

for year, month, count in rows:
label = f"{year}-{month:02d}"
labels.append(label)
values.append(count)
raw_data[label] = count
total_cves += count

# If no data was found, use default data
if not rows:
print("[Database] No timeline data found, using defaults")
return self.get_default_timeline_data(years)

return {
'labels': labels,
'values': values,
'total_cves': total_cves,
'months_covered': len(labels),
```

```
'raw_data': raw_data
}

except Exception as e:
print(f"[Database] Error fetching timeline data: {str(e)}")
return self.get_default_timeline_data(years)

finally:
cursor.close()
self.return_connection(conn)

def get_default_timeline_data(self, years=1) -> Dict[str, Any]:
"""Get default timeline data for visualization when database fails"""
labels = []
values = []
raw_data = {}
total_cves = 0

# Use default timeline data to ensure chart works
current_year = datetime.now().year
start_year = current_year - years

for (year, month), count in DEFAULT_TIMELINE_DATA.items():
if year >= start_year:
label = f"{year}-{month:02d}"
labels.append(label)
values.append(count)
raw_data[label] = count
total_cves += count

return {
'labels': labels,
'values': values,
'total_cves': total_cves,
'months_covered': len(labels),
'raw_data': raw_data
}

def save_summary_stats(self, key, count, data):
"""Save summary statistics to database with TTL for caching"""
if not self.use_database:
return False

conn = self.get_connection()
if not conn:
print(f"[Database] Failed to get connection for save_summary_stats: {key}")
return False

try:
cursor = conn.cursor()

cursor.execute("""
INSERT INTO summary_stats (key, count, last_updated, data)
VALUES (%s, %s, CURRENT_TIMESTAMP, %s)
ON CONFLICT (key) DO UPDATE SET
count = EXCLUDED.count,
last_updated = CURRENT_TIMESTAMP,
data = EXCLUDED.data
""", (key, count, json.dumps(data)))

conn.commit()
return True

except Exception as e:
print(f"[Database] Error saving summary stats: {str(e)}")
conn.rollback()
return False

finally:
cursor.close()
self.return_connection(conn)

def get_summary_stats(self, key):
"""Get summary statistics from database with TTL check"""
if not self.use_database:
# Return default data based on key
if key == 'cwe_severity':
```

```
return self.get_default_cwe_severity_stats()
return None

conn = self.get_connection()
if not conn:
print(f"[Database] Failed to get connection for get_summary_stats: {key}")
if key == 'cwe_severity':
return self.get_default_cwe_severity_stats()
return None

try:
cursor = conn.cursor(cursor_factory=RealDictCursor)

cursor.execute("""
SELECT count, last_updated, data
FROM summary_stats
WHERE key = %s
""", (key,))

row = cursor.fetchone()
if row:
# Check TTL (24 hours instead of 6 to reduce database load)
last_updated = row['last_updated']
now = datetime.now()

if isinstance(last_updated, str):
# Convert string to datetime
last_updated = datetime.fromisoformat(last_updated.replace('Z', '+00:00'))

age = now - last_updated

if age.total_seconds() < 24 * 60 * 60:  # 24 hours
return {
'count': row['count'],
'last_updated': row['last_updated'],
'data': row['data']
}
else:
print(f"[Database] Cache for {key} expired ({age.total_seconds()/3600:.1f} hours old)")

# If no data found or cache expired, return defaults based on key
if key == 'cwe_severity':
return self.get_default_cwe_severity_stats()
return None

except Exception as e:
print(f"[Database] Error getting summary stats: {str(e)}")
# Return default data based on key
if key == 'cwe_severity':
return self.get_default_cwe_severity_stats()
return None

finally:
cursor.close()
self.return_connection(conn)

def get_default_cwe_severity_stats(self):
"""Get default CWE severity stats for visualization when database fails"""
# Calculate severity counts from default data
severity_counts = {
"CRITICAL": 0,
"HIGH": 0,
"MEDIUM": 0,
"LOW": 0
}

for cwe_id, data in DEFAULT_CWE_SEVERITY_DATA.items():
severity = data["severity"]
count = data["count"]
severity_counts[severity] += count

# Format data for chart
cwe_chart_data = {
"labels": list(DEFAULT_CWE_SEVERITY_DATA.keys()),
"datasets": [
{
"severity": "CRITICAL",
```

```
"count": severity_counts["CRITICAL"],
"color": "#ff4d4d"
},
{
"severity": "HIGH",
"count": severity_counts["HIGH"],
"color": "#ffaa33"
},
{
"severity": "MEDIUM",
"count": severity_counts["MEDIUM"],
"color": "#5599ff"
},
{
"severity": "LOW",
"count": severity_counts["LOW"],
"color": "#44cc88"
}
],
"cwe_data": DEFAULT_CWE_SEVERITY_DATA
}

return {
'count': sum(severity_counts.values()),
'last_updated': datetime.now().isoformat(),
'data': cwe_chart_data
}

def get_stats(self) -> Dict[str, Any]:
"""Get database statistics for monitoring"""
if not self.use_database:
return {'database_enabled': False, 'memory_mode': True}

conn = self.get_connection()
if not conn:
print("[Database] Failed to get connection for get_stats")
return {'database_enabled': False, 'connection_failed': True}

try:
cursor = conn.cursor()

cursor.execute("SELECT COUNT(*) FROM cves")
total_cves = cursor.fetchone()[0]

# Get table sizes (if possible)
try:
cursor.execute("""
SELECT relname as table, pg_size_pretty(pg_total_relation_size(relid)) as size
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC
""")
tables = {row[0]: row[1] for row in cursor.fetchall()}
except:
tables = {}

# Check if default data is loaded
cursor.execute("SELECT COUNT(*) FROM timeline_data")
timeline_count = cursor.fetchone()[0]

cursor.execute("SELECT COUNT(*) FROM summary_stats WHERE key = 'cwe_severity'")
cwe_data_exists = cursor.fetchone()[0] > 0

return {
'total_cves': total_cves,
'database_enabled': True,
'tables': list(tables.keys()),
'table_sizes': tables,
'timeline_data_loaded': timeline_count > 0,
'cwe_data_loaded': cwe_data_exists
}

except Exception as e:
print(f"[Database] Error getting stats: {str(e)}")
return {'database_enabled': False, 'error': str(e)}

finally:
cursor.close()
```

```
self.return_connection(conn)

def close(self):
"""Close all connections in pool"""
if self.connection_pool:
self.connection_pool.closeall()
print("[Database] Connection pool closed")

# Global database manager instance
db_manager = DatabaseManager()
```